2024 lEEE 13th Data Driven Control and Learning Systems Conference
May 17-19, 2024, Kaifeng, China

Hierarchical Self-Attention Graph Pooling Networks for Leakage Detection of Water Distribution Networks

Wenchao Cui[1], Weixing Liu[2], Tong Wang[3]

1. Harbin Institute of Technology, Harbin 150001, China
E-mail: wchaocui@163.com

2. Aerospace System Engineering Shanghai, Shanghai 201108, China
E-mail: liuweixing@hit.edu.cn

3. Harbin Institute of Technology, Harbin 150001, China
E-mail: twang@hit.edu.cn

Abstract: Leakage detection in urban water supply systems is crucial for saving water resources and ensuring social production. Unlike traditional data-driven methods, the approach introduced to leak detection in this study is based on graph neural networks, which can better explore the hidden information in the topology structure of the pipeline network itself. This article first simulates the leakage of a real pipeline network using EPANET to obtain experimental data and uses a hierarchical self-attention pooling graph neural network for graph classification. The experimental results indicated that this method has certain effectiveness in large-scale water supply networks.

Key Words: GNN, self-attention graph pooling, Leakage Detection

2024 IEEE 13th Data Driven Control and Learning Systems Conference (DDCLS) | 979-8-3503-6167-4/24/$31.00 ©2024 IEEE | DOI: 10.1109/DDCLS61622.2024.10606924

1 Introduction

As an important support for people's daily lives, and social development, the safety and stability of the water supply network system are vitally significant. Leakages of water distribution networks (WDNs) not only lead to the waste of resources but also severely disrupt industrial and residential activities. Therefore, investigating the detection and localization of the leakage is a fundamental task for improving water quality, safeguarding the urban water supply, and reducing the loss of water resources. The leakage detection technologies can be classified into two categories: hardware-based detection and software-based detection.

Hardware-based detection methods rely on the technology of detection equipment. By measuring and analyzing specific indicators such as sound and vibration generated within the pipelines to achieve leakage detection. Commonly used methods include the listening stick method, correlation leak detector, leak noise loggers, infrared spectroscopy, ground-penetrating radar, pipeline internal inspection, distributed optical fiber sensing, tracer gas method, smart balls, and satellite detection [1]. All of the mentioned methods require the expertise of operators and are limited by the precision of the devices. Furthermore, these methods are highly sensitive to the constraints of detection scenarios, including factors such as the pipe material and diameter size, and thus lack universal applicability. Additionally, the accuracy of detection is directly linked to the expense of the hardware, which translates to a low cost-benefit ratio.

With the development of computer technology, software-based detection methods have made significant progress. In these approaches, operational data is collected from the pipelines, and then models or algorithms are built into software to analyze and locate leaks. The software-based methods include the balancing approach, the Minimum Night Flow (MNF) analysis, transient analysis, model-based analysis, and data-driven approaches [2], [3].

The mass balance method and MNF analysis are founded on the principle of mass conservation, where the difference between the inflow and outflow indicates the leakage. MNF specifically examines the flow from 2:00 AM to 4:00 AM. Due to these methods can only be used to identify the presence of a leakage zone the accuracy of these two kinds of methods is poor. The transient methods assess hydraulic signals within pipelines from the time domain or frequency domain to determine the leakage. A transient method proposed in [4], used the frequency response function (FRF) to analyze anti-transient pressure waves for leak detection in water-filled plastic pipes. Despite its low cost, the method is limited due to the difficulty of obtaining the initial signal, causing this method to be only suitable for small and uncomplicated pipeline networks within laboratory environments. Model-based methods construct a virtual hydraulic model using real pipeline network data, simulating various leakage scenarios for each pipe [5]. Then compare the simulated leakage data with the actual operating data to pinpoint leaks under specific conditions. These methods heavily rely on the precision of the hydraulic models, when the actual pipeline network is complex, the accuracy of leakage detection will be greatly limited.

The data-driven detection techniques can be broadly classified into two categories: statistical approaches and artificial intelligence-based approaches.

Statistical approaches utilize statistical process control (SPC) and DMA (District Metered Area) partitioning techniques to establish statistical models for detecting leaks. These techniques are based on the assumption that the random variations in the monitoring data conform to a Gaussian distribution. However, in practical applications, these methods face considerable uncertainty due to non-

*This work was supported by the National Key Research and Development Program (2022YFC3203802).

stationary changes in the data, which often deviate from the Gaussian distribution assumption [6]. Machine learning methods focus on using algorithms to extract features from datasets to classify or predict leak and non-leak situations. Fares Ali et al. [7] demonstrated the stability of these methods in detecting leaks using sound signals collected from an actual pipeline network in Hong Kong. Walt et al. [8] proposed a reverse analysis technique to detect leak locations and magnitudes using flow and pressure data. The results showed that for simple networks, Support Vector Machine (SVM) and Artificial Neural Networks (ANN) could accurately locate leaks, but with low accuracy in complex networks. Kang et al. [9] proposed a system framework that combined a CNN-SVM leak detection model with a graph-based localization algorithm. Results showed that this model had higher accuracy in leak detection compared to individual CNN and SVM models, albeit with a longer training period. In [10], a leak detection model that combined LSTM (Long Short-Term Memory) and CNN was proposed. This model selected specific pressure monitoring points using fuzzy C-means clustering and collected data. The model accuracy reached 90.25% and the loss rate reached 20.24%.

Based on the current state of research, it appears that traditional detection methods have predominantly focused on data collected from sensors, neglecting the graph relationships between nodes and pipes. Although CNN applies to Euclidean data effectively, it still has limitations in leakage detection, which possesses non-Euclidean structures. Furthermore, the performance of leakage detection in small-scale pipe networks is superior. Nonetheless, the previously cited literatures do not provide experimental validation for the effectiveness of these models on large-scale pipe networks. By applying graph learning methods, we can uncover further information for analyzing leaks from the topological structure of pipe networks. In this study, we have experimented with Hierarchical Self-Attention Graph Pooling Networks to explore leakage detection from the graph level within large-scale WDNs.

2 Simulation of Leakage and Dataset

The water distribution network chosen for simulation was provided by Lindell Ormsbee from the Water Distribution System Research Database. The network map used for the simulation of data consisted of two tanks, 1 pump, 984 pipes, 858 junction nodes, and approximately 65618 meters of pipe [11]. It is primarily a grid system in Kentucky serving 6,400 customers. This pipe network topology is shown in Fig.1.

2.1 The Principle of Leakage Simulation Based On EPANET

Leakage is correlated with node pressure, and thus the leak model can be considered a pressure-driven phenomenon [12], as shown in Equation (1).

$$Q_i = K_i P_i^n \tag{1}$$

where Q_i is the leakage flow rate at node i, K_i is the emitter coefficient, P_i is the pressure at node i, and n is the pressure exponent which primarily depends on the pipe material (typically 0.5 for metal pipes, 1.2 or greater for plastic pipes, and 1.0 for composite materials) [13].

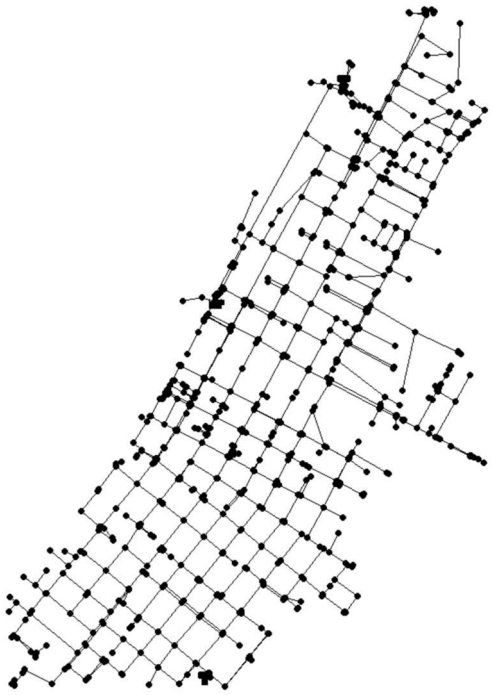


Fig. 1: Topological structure of water supply network

The simulation system was established using EPANET 2.0, a platform renowned for its high precision and reliability in hydraulic and water quality modeling. In EPANET, the emitter coefficient is an attribute of the node, allowing users to adjust the parameter to designate a node as a leakage point. The value of this parameter is directly proportional to the degree of leakage. According to the Water Balance Equation, the total input to the WDNs is equal to the actual water demand plus the leakage. Assuming an average leakage rate of $m(0<m<1)$, then the emitter coefficient parameter C can be calculated using Equation (2).

$$\begin{cases} Q_{ls} = \dfrac{m}{1-m} Q_{sj} \\ C = \dfrac{Q_{ls}}{(\dfrac{1}{n}\sum_i^n P_i)^g} \end{cases} \tag{2}$$

Where Q_{ls} represents the total leakage of the pipeline network and Q_{sj} is the actual water demand, P_i denotes the pressure at the leakage point i. By selecting n nodes as leakage simulation points, the emitter coefficient can be readjusted to emphasize the differences among the leakage nodes by assigning weights. The weight value μ_i can be determined based on the length, diameter, and material of the pipes connected to the nodes [14]. The parameter C_i for simulation point i can then be calculated using Equation (3).

$$C_i = m_i C \tag{3}$$

Based on the provided pipe network data, which indicates a leakage rate of 21%, we have identified and configured five nodes to simulate the occurrence of leaks. At each of these nodes, the coefficient parameter has been

assigned distinct values to accurately represent varying severities of leakage.

2.2 Datasets

Considering the varying water usage patterns across different regions, water distribution networks (WDNs) have been categorized into residential, commercial, and industrial areas. Under these different modes, peak consumption varies significantly. In general, residential areas exhibit peak water usage around 8 AM, noon (12 PM), and 8 PM. The commercial districts experience heightened water demand around 11 AM and 10 PM. In contrast, the industrial sector sees a significant increase in water consumption post-8 AM, with another notable peak around 5 PM. The pattern editor in EPANET enables the simulation of water usage patterns over any desired period length. Within a 24-hour time-delay cycle, each hour is treated as a simulation step, and each step is multiplied by a factor with a unique value. The average of these factors equals one. These factors are positively correlated with water consumption, meaning that the actual water consumption during each period is calculated as the product of the basic water demand and the respective multiplier. The comparative illustration of three distinct patterns across a 24-hour cycle is presented in Fig.2.

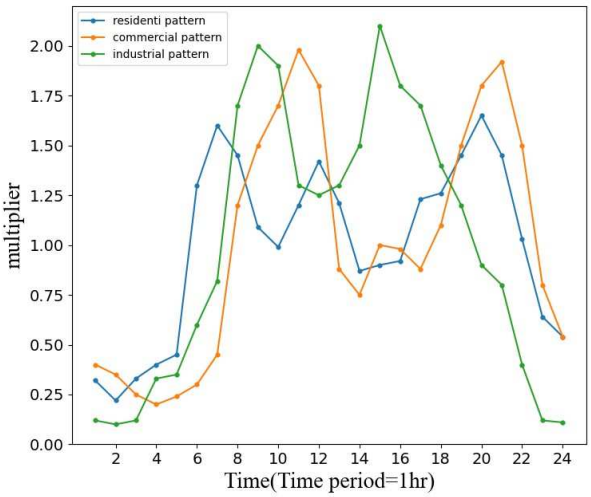


Fig. 2: Line chart of multipliers in three patterns

In the Water Distribution Networks (WDNs), five nodes have been evenly distributed to serve as leakage points, with their emitter coefficients calculated using the formula (3). Setting the parameter of all nodes to zero can achieve the transition from the leakage state to the regular state. The pipe flow and node pressure vary with diverse leak conditions. Including the normal condition, each of the conditions is labeled and the detection datasets are divided into six distinct categories. After the extended period analysis of those six simulation modes, use Python to get the dataset generated by the EPANET. To construct a comprehensive dataset, continuous 5-day node data has been collected.

Due to the specific structure of the WDNs, we processed the dataset into graph data. Graph data is a special data structure consisting of vertexes and edges, each vertex is a symbol of an object, and the edges represent the relationships between objects. We considered the nodes and pipes in the water supply network as vertexes and edges separately, each node has four-dimensional features. The dataset consists of 668,480 nodes, 708,480 edges, and 720 graphs. Utilize the simulation step as the data collection interval, each graph is the detection data of all nodes at a specific moment. The graph dataset of time series can be shown in Fig.3.

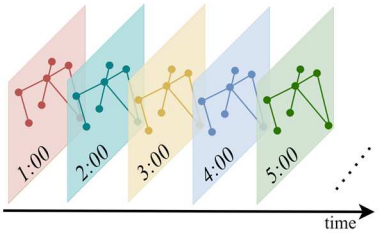


Fig. 3: The graph dataset of time series

Frame the leakage detection challenge as a graph classification task, assigning a label to each graph to indicate its status. Before the experiment, we determined the nature of the graph data we would construct. Given the unidirectional flow of the pipes, the edges of the graph are directed. Additionally, since the features and structure remain constant over time, the graph we have established is classified as a directed homogeneous static graph.

3 Hierarchical Self-Attention Graph Pooling Networks

3.1 Graph Neural Network

The concept of graph neural networks (GNNs) was initially articulated in 2005 [15], with the primary objective of enabling the direct processing of graph-structured data while maintaining topological dependency of the information on the nodes. Initially, research concentrated on iterative approaches and the extension of recursive neural networks, which are classified as Recurrent Graph Neural Networks (RecGNNs) [16]. However, RecGNNs were limited by their high computational costs. To address CNNs not suited for non-Euclidean domains, Bruna et al. from spectral and spatial domains to extend the application of CNNs [17]. In 2016, Defferrard et al. proposed the ChebNet, which approximated the convolutional kernel using Chebyshev polynomials of degree k [18]. Kipf et al. further refined the concept with the Graph Convolutional Network (GCN), which utilized only the first-order Chebyshev polynomial and reduced the convolution kernel to a single parameter [19]. Designed for static graphs, spectral graph convolution does not support dynamics or evolving graph structures. Furthermore, the convolution kernel is determined by the eigenvectors of the Laplacian matrix, which are fixed for a given graph. This lack of flexibility limits the ability to adapt to different types of graph structures or tasks.

In spatial-based convolutional GNNs, the convolutional filters are applied directly to the graph nodes and their neighbors, offering a dynamic and adaptable approach to graph-based learning. Given that the datasets established were the directed graphs, we adopted the spatial convolution operations for the feature extraction. Similar to

CNNs, the spatial-based graph convolutions convolve the central node's representation with its neighbors' representations to derive the updated representation for the central node, as shown in Fig.4.

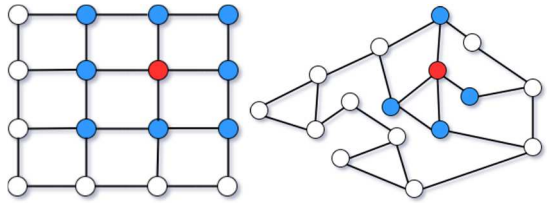


Fig. 4: 2-D convolution and graph convolution

3.2 Hierarchical pooling structure

In CNNs, pooling layers are instrumental in reducing the dimensionality of feature maps by down-sampling, which helps prevent overfitting. This principle is similarly applied in graph pooling methods to manage computational complexity. Pooling can be broadly categorized into two types: global pooling and hierarchical pooling [20]. As a readout mechanism, global pooling equally aggregates node features after a fixed number of iterations, but it overlooks the hierarchical representations for capturing graph structure. To address this, we implemented hierarchical pooling to learn features and topological information at each layer.

To facilitate graph classification by leveraging node features, we propose employing the top-k pooling method, also known as the graph pooling (gPool) layer. This approach involves selecting the top k nodes, ranked by their importance, to reduce the graph to a smaller size [21]. Assuming a graph with N nodes, each with C features, the entire process can be described by the following equations:

$$idx = rank(z, k) \tag{4}$$

$$z = X^{\ell} p^{\ell} / \left\| p^{\ell} \right\| \tag{5}$$

$$\tilde{X}^{\ell} = X^{\ell}(idx, :) \tag{6}$$

$$A^{\ell+1} = A^{\ell}(idx, idx) \tag{7}$$

$$X^{\ell+1} = \tilde{X}^{\ell} \odot \tanh(z) \tag{8}$$

where k is the selection ratio of nodes remaining in the new graph. The function $rank(z, k)$ returns indices of the k-largest values in vector z. Here, p is a trainable projection vector, and z is the scalar projection value on p for ranking. $\tilde{X}^{\ell} \in R^{N \times C}$ and $A^{\ell+1} \in R^{N \times N}$ is the row or column extraction to form the feature matrix and the adjacency matrix for the new graph [22]. To effectively fuse the information of all nodes, a global pooling and maximum pooling readout mechanism is added after each gPool layer, as shown in Equation (9).

$$s = \frac{1}{N} \sum_{i=1}^{N} x_i \,\|\, \max_{i=1}^{N} x_i \tag{9}$$

3.3 Hierarchical Self-Attention Graph Pooling Model

Given the adjacency matrix A and the feature matrix X, the GCN captures the spatial features between nodes by considering their first-order neighborhood. Subsequently, a GCN model can be constructed by stacking multiple convolutional layers, as described by Equation (10) [19].

$$X_{\ell+1} = \sigma(\hat{D}^{-\frac{1}{2}} \hat{A} \hat{D}^{-\frac{1}{2}} X_{\ell} W_{\ell}) \tag{10}$$

where σ is the activation function, $\tilde{A} \in R^{N \times N}$ is the adjacency matrix with self-connections, $\tilde{D} \in R^{N \times N}$ is the degree matrix of the adjacency matrix, $X \in R^{N \times C}$ is the input features of the graph with N nodes and C dimensional features, W is a trainable weight matrix that applies a linear transformation to feature vectors.

Attention mechanisms enable models to focus on important features while downplaying less significant ones. Self-attention, in particular, excels at addressing long-distance dependencies, a challenge inherent in traditional recurrent neural networks (RNNs) and CNNs. It allows the model to directly concentrate on the relationships between any two positions within a sequence without sequence length constraints [23]. The importance score of each node can be calculated using Equation (11).

$$Z = \sigma(\hat{D}^{-\frac{1}{2}} \hat{A} \hat{D}^{-\frac{1}{2}} X \Theta_{att}) \tag{11}$$

The importance score Z is dynamically learned through graph convolution, with the weight parameter Θ_{att} being the unique parameter of the self-attention mechanism as described in [24], [25]. The top-k pooling operation is then executed, prioritizing nodes based on their calculated importance scores. This process filters out nodes deemed less significant, thereby refining the adjacency matrix and updating the node features to retain only the most relevant information. After a series of convolutional attention pooling operations, a readout mechanism is applied to amalgamate the outputs from each layer. This mechanism continues to utilize the operations defined by Formula 9. The comprehensive model architecture is illustrated in Figure 5.

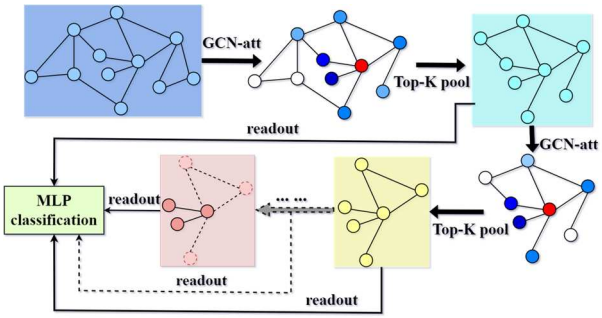


Fig. 5: The hierarchical self-attention graph pooling model

4 Experiments and Analysis

4.1 Experiment on the Leakage Detection

The proposed model is tested on the datasets simulated by EPANET as described in section II. To reframe the leakage detection as a graph classification task, we constructed graph data for each detection instance to encapsulate the dynamic information across temporal dimensions. The datasets are obtained through multiple simulations under situations of different leakage locations and different amounts of leakage, encompassing the node attributes, adjacency matrices, and graph labels. The node

attributes consist of labels, pressure, detection time, and unique identifiers. The node labels categorize areas into residential, commercial, industrial, and miscellaneous, while unique IDs correspond to each node's geographical coordinates in the pipeline network.

We have configured our model with four layers of convolutional pooling, each with a distinct retention rate. Given the substantial number of nodes in the graph, the first two pooling layers are intentionally designed with a low retention rate to filter out irrelevant nodes and expedite the training process. Four readout layers have been established to extract and aggregate feature representations employing a hybrid approach of maximum and average pooling. The dataset was apportioned into three segments with a ratio of 8:1:1, corresponding to the training set, validation set, and test set, respectively.

4.2 Evaluation Metrics

To evaluate the model's classification performance, four critical metrics have been selected. Assuming that the correct category label is denoted as positive (*P*) and the incorrect one as negative (*N*), the following definitions apply: *TP* (True Positive) indicates the instances correctly identified as positive, *TN* (True Negative) denotes the instances correctly identified as negative, *FP* (False Positive) signifies the instances incorrectly labeled as positive, and *FN* (False Negative) refers to the instances incorrectly labeled as negative.

The accuracy metric measures the ratio of correctly classified instances across all categories relative to the total dataset, as detailed in Equation (12).

$$Acc = \frac{TP + TN}{TP + FN + TN + FP} \tag{12}$$

Precision is the metric that assesses the proportion of true positives among all instances classified as positive, with the formula for each category given by Equation (13),

$$P_i = \frac{TP_i}{TP_i + \sum_{j \neq i} FP_j} \tag{13}$$

where FP_j denotes the instances where category j is mistakenly identified as category i.

Recall, or sensitivity, indicates the proportion of actual positives correctly identified within the set of positives, with the formula for each category provided by Equation (14),

$$R_i = \frac{TP_i}{TP_i + \sum_{j \neq i} FN_j} \tag{14}$$

where FN_j represents instances where category i is incorrectly classified as category j.

The F1-score is a composite measure of precision and recall, and its calculation is as follows:

$$F1-score = \frac{2 \times P_i \times R_i}{P_i + R_i} \tag{15}$$

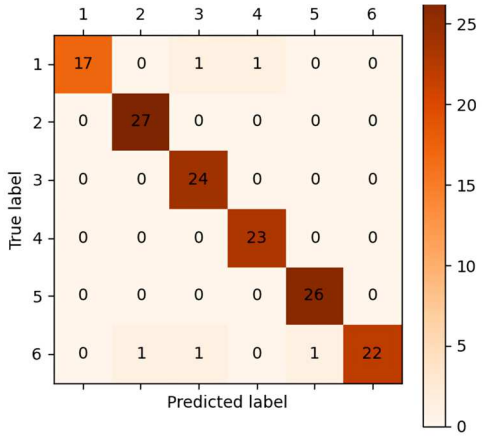
4.3 Results

By employing the model we proposed, which is based on graph classification, we have achieved an approximate accuracy rate of 92% in successfully detecting leaks within the test dataset. Table 1 delineates the model's performance metrics over a specific training epoch, where labels 1 - 6 correspond to the absence of leakage and five distinct categories of leaking nodes, respectively.

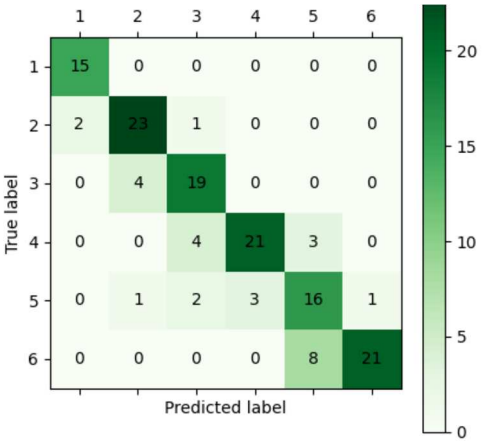
Table 1: The classification performance of the model

Leakage label	Precision	Recall	F1-score	Support
1	1.00	0.89	0.94	19
2	0.96	1.00	0.98	27
3	0.92	1.00	0.96	24
4	0.96	1.00	0.98	23
5	0.96	1.00	0.98	26
6	1.00	0.88	0.94	25

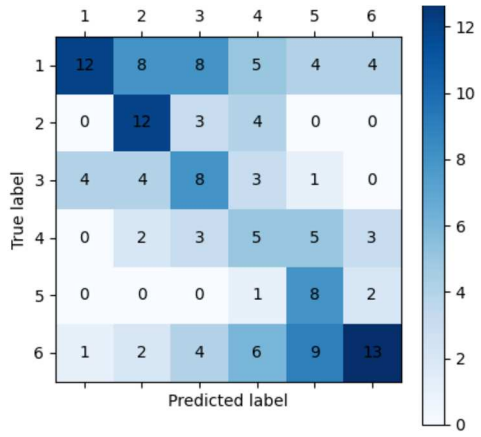
The confusion matrix displayed in Fig.6, which illustrates the leak detection performance of three models: the hierarchical self-attention pool model, the global self-attention pool model, and the GCN-only model. The results demonstrate the substantial efficacy of the proposed model in detecting leaks. The self-attention pooling mechanism is particularly effective in the leak detection task.



(a) confusion matrix of hierarchical self-attention pool model



(b) confusion matrix of global self-attention pool model



(c) confusion matrix of GCN-only model

Fig. 6: The confusion matrix of the leakage detection models

5 Conclusions

In this paper, we have proposed a leakage detection method utilizing the graph neural networks on a WDNs created by the EPANET software. The experiments verified the feasibility of graph neural networks in leak detection of large-scale water supply system, and found that graph convolution based on self-attention graph pooling mechanism can effectively extract information from data of multi node graph structures. However, there are differences in the detection and recognition ability of the model for leakage at different positions. In the future, the model will be further improved to enhance its leakage detection ability.

References

[1] El-Zahab S, Zayed T, Leak detection in water distribution networks: an introductory overview, *Smart Water*, 4(1): 1-23, 2019.

[2] T. K. Chan, C. S. Chin, X. Zhong, Review of Current Technologies and Proposed Intelligent Methodologies for Water Distributed Network Leakage Detection, *IEEE Access*, 12(3): 6, 2018.

[3] Javad S, Hassan S H, Masoud S, Computational methods for pipeline leakage detection and localization: A review and comparative study, *Journal of Loss Prevention in the Process Industries*, (pre-publish):104771-, 2022.

[4] Bin Pan, Huan-Feng Duan, Silvia Meniconi, Bruno Brunone, FRF-Based Transient Wave Analysis for the Viscoelastic Parameters Identification and Leak Detection in Water-Filled Plastic Pipes, *Mechanical Systems and Signal Processing*, 146, 2021.

[5] Fan X, Zhang X, Yu X B, Machine learning model and strategy for fast and accurate detection of leaks in the water supply network, *Journal of Infrastructure Preservation and Resilience*, 2(10): 1-21, 2021.

[6] Yipeng Wu, Shuming Liu, A review of data-driven approaches for burst detection in water distribution systems, *Urban Water Journal*,14(9): 972-983, 2017.

[7] Fares Ali, Tijani IA, Rui Zhang, Leak Detection in Real Water Distribution Networks Based on Acoustic Emission and Machine Learning, *Environmental Technology*, 22(1): 25,2022.

[8] J.C. Walt, P.S. Heyns, D.N. Wilke, Pipe Network Leak Detection: Comparison Between Statistical and Machine Learning Techniques, *Urban Water Journal*, 15(10): 23, 2019.

[9] Kang J, Park Y J, Lee J, et al, Novel leakage detection by ensemble CNN-SVM and graph-based localization in water distribution systems, *IEEE Transactions on Industrial Electronics*, 65(5): 4279-4289, 2017.

[10] X. Wu and C. Zhang, Leakage Location Method of Water Supply Pipe Network Based on Integrated Neural Network, in *IEEE Asia-Pacific Conference on Image Processing, Electronics, and Computers (IPEC)*, Dalian, China, 2022: 670-675.

[11] Water distribution system research database, Website , 2016,http://www.uky.edu/WDST/database.html.

[12] Dina Z, et al, Exploring the key facets of leakage dynamics in water distribution networks: Experimental verification, hydraulic modeling, and sensitivity analysis, *Journal of Cleaner Production*, 362,2022.

[13] J. Kemba, K. Gideon, C. N. Nyirenda, Leakage Detection in Tsumeb East Water Distribution Network Using EPANET and Support Vector Regression, in *2017 IST-Africa Week Conference (IST-Africa)*. Windhoek, Namibia: IEEE, 2017: 1-8.

[14] Cobacho R, Arregui F, Soriano J, et al, including leakage in network models: an application to calibrate leak valves in EPANET, *Journal of Water Supply: Research and Technology-Aqua*, 64(2): 130-138, 2015.

[15] M. Gori, G. Monfardini and F. Scarselli, A new model for learning in graph domains, Proceedings, in *2005 IEEE International Joint Conference on Neural Networks*, 2005, Montreal, QC, Canada, 2005: 729-734

[16] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner and G. Monfardini, The Graph Neural Network Model, *IEEE Transactions on Neural Networks*, 20(1): 61-80, 2009.

[17] J. Bruna, W. Zaremba, A. Szlam, and Y. LeCun, Spectral networks and locally connected networks on graphs, in *Proc. ICLR*, 2014: 1–14.

[18] M. Defferrard, X. Bresson, and P. Van der Gheynst, Convolutional neural networks on graphs with fast localized spectral filtering, in *Proc. NIPS*, 2016: 3844–3852.

[19] T. N. Kipf and M. Welling, Semi-supervised classification with graph convolutional networks, in *Proc. ICLR*, 2017: 1–14.

[20] Z. Wu, S. Pan, F. Chen, et al, A comprehensive Survey on Graph Neural Networks, *IEEE Transactions on Neural Networks and Learning Systems*, 32(1): 4-24, 2022.

[21] Cangea, Cătălina, et al, Towards sparse hierarchical graph classifier, *arXiv preprint arXiv:1811.01287*,2018.

[22] Gao, Hongyang, and Shuiwang Ji, Graph u-nets, in *International conference on machine learning*. PMLR, 2019: 2083-2092.

[23] Vaswani A, Shazier N, Parmar N, et al, Attention is all you need, *Advances in neural information processing systems*, 30, 2017.

[24] Lee, Junhyun, Inyeop Lee, and Jaewoo Kang, Self-attention graph pooling, in *International conference on machine learning. PMLR*, 2019: 3734-3743.

[25] Ying, Zhitao, et al, Hierarchical graph representation learning with differentiable pooling, *Advances in neural information processing systems*, 31,2018.